



Universidad de Granada

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y
MATEMÁTICAS

TÉCNICAS DE LOS SISTEMAS INTELIGENTES

Apuntes de Clase

Autor:
Jesús Muñoz Velasco

Curso 2025-2026

Índice general

1. Tema 1	5
1.1. Algoritmos. Árbol de Estados.	5
1.2. Algoritmo A^*	6
1.2.1. Completitud de A^*	7
1.2.2. Admisibilidad de A^*	8
1.2.3. Dominancia	9
1.2.4. Heurísticas monótonas	10

1. Tema 1

1.1. Algoritmos. Árbol de Estados.

Definición 1.1. Un algoritmo A es una secuencia ordenada, finita y bien definida de pasos o instrucciones lógicas que permiten resolver un problema.

Definición 1.2. Diremos que un algoritmo A es **completo** si encuentra una solución al problema en caso de que exista en un tiempo finito.

Definición 1.3. Diremos que un algoritmo A es **admisible** si, en caso de encontrar solución a un problema, esta es óptima.

Definición 1.4. Dados dos algoritmos A_1 y A_2 , diremos que A_2 **domina** a A_1 si todo nodo n extendido por A_1 es también expandido por A_2 . En el caso de que A_2 domine tendremos que será más eficiente en el tiempo de búsqueda.

Notación. A continuación describiremos la notación que usaremos a lo largo de estos apuntes:

n	Nodo cualquiera.
s	Estado inicial.
γ	Nodo objetivo .
Γ	Conjunto de objetivos .
Γ^*	Conjunto de nodos a los que se puede llegar con un camino óptimo.
$P_{n_i-n_j}$	Camino arbitrario del nodo n_i al nodo n_j .
$P_{n_i-n_j}^*$	Camino óptimo del nodo n_i al nodo n_j .
$P_{n_i-\Gamma}$	Camino arbitrario de n_i a un objetivo .
c^*	Costo óptimo de s a una solución admisible.

Definimos además las funciones que más utilizaremos:

$k(n_i, n_j)$	Coste óptimo entre n_i y n_j
$g(n)$	Costo actual desde s hasta n .
$g^*(n)$	Costo óptimo entre s y n , es decir, $g^*(n) = k(s, n)$ para todo n .
$g_P(n)$	Costo desde s hasta n por el camino P .
$h(n)$	Costo estimado del nodo n al objetivo γ .
$h_P(n)$	Costo estimado del nodo n al objetivo γ por el camino P .
$f(n)$	Costo total estimado al objetivo γ pasando por el nodo n ($f(n) = g(n) + h(n)$).

Propiedades. Hay dos propiedades de los árboles de estados que están siempre presentes:

1. El grafo es **localmente finito**, es decir, el número de sucesores de cada nodo es finito.
2. Se verifica que $c(n_i, n_j) \geq \delta > 0$ para cualesquiera n_i, n_j nodos, es decir, el costo de cambiar de nodo es siempre positivo.

Observación. Un árbol de estados puede tener varios nodos iniciales $s_1, \dots, s_n$. Para simplificar el problema lo que se hace es crear un estado inicial artificial s y se conecta a cada uno de los estados iniciales $s_1, \dots, s_n$.

También puede haber varios nodos objetivo $\gamma_1, \dots, \gamma_m \in \Gamma$, sin embargo, en este caso no se puede simplificar a un único estado final.

El problema por tanto será ir de un único estado inicial s a alguno de los estados finales $\gamma \in \Gamma$.

1.2. Algoritmo A^*

Este algoritmo utiliza como función de estimación $f(n) = g(n) + h(n)$ para todo nodo n . Por la naturaleza del problema se verificará que $g(s) = 0$ y $h(\gamma) = 0$. Además, se verifica que

$$f^*(n) = g^*(n) + h^*(n)$$

Donde se verificará también

$$f^*(s) = h^*(s) = g^*(\gamma) = f^*(\gamma) = c^* \quad \forall \gamma \in \Gamma$$

En general, por la optimalidad se tendrá que, para cualquier camino P se tiene que

$$\begin{aligned} g_P(n) &\geq g^*(n) \\ h_P(n) &\geq h^*(n) \end{aligned}$$

Propiedades. Queremos estudiar ahora 2 propiedades del algoritmo A^* :

1. $f^*(n) = c^*$ para todo $n \in P_{s-\Gamma}^*$.

Demostración. Sea $n \in P_{s-\Gamma}^*$, entonces tenemos que existe un camino P tal que

$$g_P(n) + h_P(n) = c^*$$

Por la optimalidad se tendrá que

$$g^*(n) + h^*(n) \leq c^*$$

y necesariamente se tendrá que dar la igualdad, ya que en caso contrario c^* no sería óptimo y llegaríamos a contradicción. $\square$

2. $f^*(n) > c^*$ para todo $n \notin P_{s-\Gamma}^*$.

Demostración. Supongamos que $f^*(n) \leq c^*$. Entonces tendremos que

$$g^*(n) + h^*(n) \leq c^*$$

luego existe un camino P' tal que $g_P(n) + h_P(n) \leq c^*$. En caso de que se de la igualdad tendremos que P' sería óptimo y tendríamos que $n \in P_{s-\Gamma}^*$. En caso de ser menor tendríamos que c^* no es el coste mínimo, lo que también nos llevaría a contradicción. $\square$

1.2.1. Completitud de A^*

En esta sección probaremos que para un grafo localmente finito el algoritmo A^* es completo. Para ello dividiremos la demostración en 3 etapas:

Etapas 1) Si suponemos que el grafo es finito¹ entonces el algoritmo A^* termina. Esto se ve fácilmente, ya que si el algoritmo entrase en ciclos no finalizaría, sin embargo, al ser el coste positivo en cada paso, el algoritmo ignora los ciclos.

De esta forma, cuando el algoritmo explora un nodo, realmente lo que explora es un camino distinto. Como el número de caminos es finito, el algoritmo termina.

Etapas 2) Seguimos suponiendo que el grafo es finito, pero ahora queremos ver que A^* es completo, es decir, que si existe una solución la encontrará.

Demostración. La demostración la haremos por reducción al absurdo. Para ello, supongamos que A^* no encuentra la solución. Entonces tendremos que

$$ABIERTOS = \emptyset$$

Esto ocurrirá si y solo si el nodo que se está explorando no tiene hijos, pero esto nos lleva a contradicción, ya que existe un camino $P_{s-\gamma}$ (existe la solución), luego todos los nodos tienen al menos un descendiente hasta llegar a γ . $\square$

Etapas 3) Ahora ya estamos en condiciones de probar el siguiente teorema (asumiendo las 2 propiedades estudiadas previamente):

Teorema 1.1 (Completitud de A^*). Si el grafo es localmente finito, entonces A^* es completo.

Demostración. De nuevo la demostración la haremos por reducción al absurdo. Supongamos así que existe un camino $P_{s-\Gamma}$ y A^* no lo encuentra. En dicho caso, en $ABIERTOS$ siempre habrá un nodo de ese camino n , con un cierto $f(n) \in \mathbb{R}$. Como no encuentra la solución tendremos dos posibilidades:

- A^* termina. Usando la demostración de las etapas 1 y 2 llegaremos a contradicción.
- A^* no termina. Este será el caso que seguiremos desarrollando.

¹un grafo es finito si el número de caminos acíclicos es finito.

En caso de que A^* no termine significará que está infinitamente buscando. Usaremos ahora una propiedad muy útil: en un grafo localmente finito, el número de caminos de longitud finita es finito.

Por tanto, si no termina significará que se ha puesto a buscar caminos de longitud no finita, en cuyo caso

$$g(n) \text{ no finito} \Rightarrow f(n) = g(n) + h(n) \text{ no finito}$$

Esto es contradictorio con que el nodo n tenga $f(n)$ finito, ya que A^* nunca exploraría nodos con f no finita habiendo algún nodo $n \in ABIERTOS$ con $f(n)$ finito. $\square$

1.2.2. Admisibilidad de A^*

Definición 1.5. Una heurística h se dice admisible si y solo si

$$h(n) \leq h^*(n) \forall n$$

En esta sección trataremos de probar que si h es admisible entonces A^* es admisible.

Lema 1.2. Para cualquier nodo n que no esté en *CERRADOS* y para cualquier camino óptimo P entre s y n siempre se puede seleccionar un nodo n' perteneciente a P y a *ABIERTOS* tal que

$$g(n') = g^*(n')$$

Demostración. Realizaremos una demostración constructiva. Para ello comenzamos tomando un nodo $n \notin CERRADOS$ y consideramos el camino recorrido hasta dicho nodo numerándolo de la siguiente forma:

$$P = \{n_0 = s, n_1, \dots, n_k = n\}$$

Tenemos entonces 2 casuísticas posibles:

-) Si $s \in ABIERTOS$, entonces $g(s) = g^*(s) = 0$ y se cumple.
-) Si $s \notin ABIERTOS$, entonces construimos

$$\Delta = \{m \in (P \cap CERRADOS) : g(m) = g^*(m)\}$$

Tendremos que $\Delta \neq \emptyset$ ya que $s \in \Delta$. Tomamos $n^* \in \Delta$ con mayor subíndice. Sea n' el sucesor de n^* en P , se verificará que

$$g(n') \leq g(n^*) + c(n^*, n') = g^*(n^*) + c(n^*, n') = g^*(n')$$

Además, como en general $g^*(n') \leq g(n')$ tendremos la igualdad. Además, como $n' \in ABIERTOS$ tendremos que $n' \notin \Delta$. Como $n^* \in CERRADOS$ tendremos que $n' \in P$.

$\square$

Lema 1.3. Supongamos h admisible y A^* no ha terminado todavía. Entonces, para cualquier óptimo entre s tendremos que

$$\exists n' \in (P \cap ABIERTOS) : f(n') \leq c$$

Demostración. Consideramos $P_{s-\gamma}$. Como el algoritmo no ha terminado tendremos $\gamma \notin CERRADOS$ por lo que podemos aplicar el lema 1.2. Tomamos así $n' \in (P \cap ABIERTOS)$ con $g(n') = g^*(n')$. De esta forma

$$f(n') = g(n') + h(n') = g^*(n') + h(n') \stackrel{(*)}{\leq} g^*(n') + h^*(n') = f^*(n') = c^*$$

donde en $(*)$ hemos aplicado que h es admisible. Tenemos por tanto $f(n') \leq c^*$. $\square$

Teorema 1.4. Si h es admisible, entonces A^* es admisible.

Demostración. Lo haremos por reducción al absurdo, suponiendo que A^* no es admisible. Entonces el algoritmo encuentra una solución $t \in \Gamma$ que no es óptima, es decir,

$$f(t) > c^*$$

Nos colocamos en el paso anterior a que termine:

$$t \in ABIERTOS \quad t \text{ el mejor nodo, es decir, } f(t) \leq f(n) \quad \forall n \in ABIERTOS$$

Aplicando el lema 1.3 tendremos que existe un $n' \in ABIERTOS$ con $f(n') \leq c^*$ de donde

$$f(n') \leq c^* < f(t)$$

lo que nos lleva a contradicción, ya que t era el mejor nodo. $\square$

1.2.3. Dominancia

Si consideramos $h \cong 0$ tendremos que h será admisible y diremos que el algoritmo de búsqueda que use la heurística h será no informado.

Definición 1.6. Sean h_1, h_2 dos heurísticas admisibles. Decimos que h_2 está **más informada** que h_1 si $h_2(n) > h_1(n)$ para cualquier nodo n .

Teorema 1.5. Si A_2^* está más informado que A_1^* entonces A_2^* domina a A_1^* .

Observación. Algunas observaciones que podemos extraer son las siguientes:

-) Cuanto más se parezca h a h^* más eficiente va a ser la búsqueda.
-) Fuera del rango de la admisibilidad no hay garantías de encontrar la solución óptima.
-) La dominancia está relacionada con la eficiencia.

Una forma típica de implementar A^* es dándole un peso a la función h , definiendo f como

$$f(n) = g(n) + wh(n)$$

donde $w \in \mathbb{R}$. Para $w = 2$ le damos información de más (podemos perder la admisibilidad).

1.2.4. Heurísticas monótonas

Por la desigualdad triangular tenemos que $k(s, \gamma) \leq k(s, n) + k(n, \gamma)$ para cualquier nodo n , cualquier nodo inicial s y final γ . Tenemos por tanto que

$$h^*(s) \leq k(s, n) + h^*(n) \Rightarrow h^*(n) \leq k(n, n') + h^*(n') \quad \forall n, n' \text{ nodos}$$

Definición 1.7. Diremos que h es **consistente** si se verifica que

$$h^*(n) \leq k(n, n') + h^*(n') \quad \forall n, n' \text{ nodos}$$

Definición 1.8. Diremos que h es **monótona** si

$$h(n) \leq c(n, n') + h(n') \quad \forall n, \quad \forall n' \in \text{suc}(n)$$

Proposición 1.6. Una heurística h es consistente si y solo si es monótona.

Teorema 1.7. Toda heurística consistente (monótona) es admisible

Demostración. Por ser h consistente se tiene que verificar la desigualdad

$$h^*(n) \leq k(n, n') + h^*(n') \quad \forall n, n' \text{ nodos}$$

Si tomamos $n' = \gamma \in \Gamma^*$ se tiene que

$$h(n) \leq k(n, \gamma) + h(\gamma) \Rightarrow h(n) \leq h^*(n) + 0 = h^*(n)$$

para cualquier nodo n lo cual nos dice que h es admisible. $\square$

Teorema 1.8. Un algoritmo A^* guiado por una heurística monótona alcanza el camino óptimo a todos los nodos expandidos, es decir,

$$g(n) = g^*(n) \quad \forall n \in \text{CERRADOS}$$

Demostración. A^* selecciona como mejor nodo n . Supongamos que $g(n) > g^*(n)$, entonces

$$\text{ABIERTOS} = \{\dots, n, \dots\}$$

Por el lema 1 tenemos que $\exists n' \in \text{ABIERTOS}$ tal que $u' \in P_{s-n}^*$, en cuyo caso, $g(n') = g^*(n')$. Tenemos entonces

$$\begin{aligned} f(n) = g(n') + h(n') &\stackrel{\text{lema 1}}{=} g^*(n') + h(n') \stackrel{\text{consist.}}{\leq} g^*(n') + k(n', n) + h(n) \\ &\stackrel{\text{opt.}}{=} g^*(u) + h(n) \stackrel{\text{hip.}}{<} g(n) + h(n) = f(n) \end{aligned}$$

y llegamos a contradicción. $\square$

Observación. Si la heurística no es monótona no hay que revisar *CERRADOS*, alcanzamos eficiencia y el concepto de dominancia se basa en el tiempo.